Wydział Matematyki i Nauk Informacyjnych

POLITECHNIKA WARSZAWSKA

Inżyniera Oprogramowania 1

Specyfikacja

Aplikacja Pull-app

Autorzy
Franciszek Niedźwiecki
Mikołaj Matałowski
Wiktor Pańczak
Maksymilian Majewski
Igor Oleksy

15 marca 2026

Spis treści

1 Cel aplikacji **2**

2 Aktorzy w aplikacji **2**
2.1 Użytkownik 2
2.1.1 Użytkownik - Kierowca 3
2.1.2 Użytkownik - Pasażer 4
2.2 Administrator 5

3 Funkcjonalności dostarczane przez aplikację **5**
3.1 Serwer 5

4 Klasy **7**
4.1 User 7
4.2 Trip 8
4.3 Report 9
4.4 Message 10
4.5 Review 11

5 Diagram stanów **11**
5.1 Użytkownik 11
5.2 Oferta 12
5.3 Zgłosznie 13

6 Komunikacja w systemie **14**
6.1 Sekwencje wymiany wiadomości przy zadanym rodzaju działania 14
6.1.1 Proces logowania użytkownika 14
6.1.2 Publikacja nowego przejazdu 15
6.1.3 Wyszukiwanie ofert 15
6.1.4 Rezerwacja miejsca (Zgłoszenie) 16
6.1.5 Akceptacja pasażera 16
6.1.6 Wymiana wiadomości 17
6.1.7 Pobieranie aktualnych przejazdów 17
6.1.8 Historia przejazdów 18
6.2 Endpointy API serwera 19
6.3 Struktura komunikatów (DTO) 19

1 Cel aplikacji

Celem aplikacji Pull-app jest stworzenie wirtualnego rynku współdzielonych przejazdów, na którym kierowcy publikują planowane trasy i udostępniają wolne miejsca w samochodzie, a pasażerowie mogą w prosty sposób dołączać do wybranych przejazdów.

Rozwiązanie ma wspierać ograniczanie kosztów transportu (m.in. poprzez podział kosztów paliwa), zwiększać efektywność wykorzystania dostępnych miejsc w samochodach oraz stanowić wygodną alternatywę dla podróżowania w pojedynkę. Dodatkowym celem jest ułatwienie organizacji przejazdu: od wyszukania oferty i kontaktu między użytkownikami, przez ustalenie szczegółów (miejsce zbiórki, godzina, zasady rozliczeń), aż po potwierdzenie udziału.

Aplikacja ma także podnosić bezpieczeństwo i jakość korzystania z usługi poprzez mechanizmy weryfikacji użytkowników, system ocen i opinii oraz przejrzyste przedstawienie informacji o kierowcy.

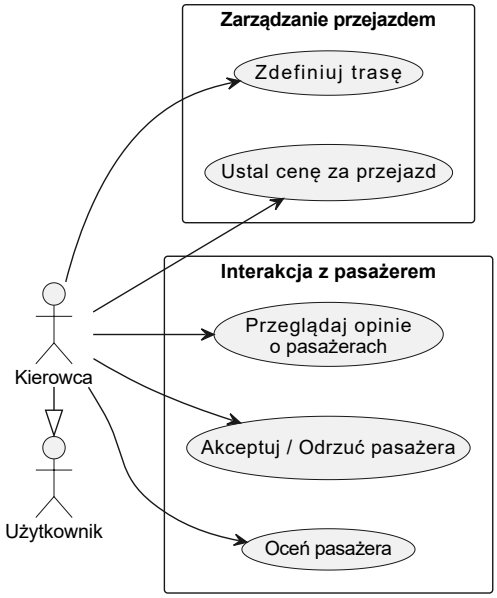
2 Aktorzy w aplikacji

2.1 Użytkownik

- Użytkownik może stworzyć konto, aby korzystać z aplikacji
- Użytkownik może logować się i wylogować się, aby mieć bezpieczny dostęp do prywatnych danych opublikowanych na platformie
- Użytkownik może resetować hasło, aby nie stracić dostępu do konta
- Użytkownik może zgłaszać błędy, aby administratorzy mogli je naprawić
- Użytkownik może modyfikować dane prywatne na swoim profilu, aby inni użytkownicy mogli je zobaczyć
- Użytkownik może wysyłać oraz odbierać wiadomości od innych użytkowników

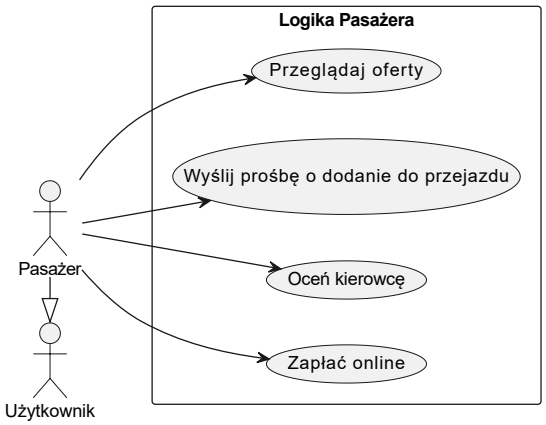
2.1.1 Użytkownik - Kierowca

- Kierowca może publikować swoją trasę, aby znaleźć współpasażerów do podróży
- Kierowca może akceptować pozostałych pasażerów, aby wyznaczać grupę do wspólnej podróży
- Kierowca może wyświetlić dane pasażera, aby wiedzieć z kim będzie podróżować
- Kierowca może wyświetlać opinie o pasażerze, aby podjąć decyzję o akceptacji pasażera
- Kierowca może wyznaczyć wysokość składki za paliwo na przejazd, aby oszczędzić koszt podróży oraz zachęcić potencjalnych pasażerów
- Kierowca może wystawić opinię o pasażerze, aby inni pasażerowie oraz kierowcy wiedzieli więcej o pasażerze



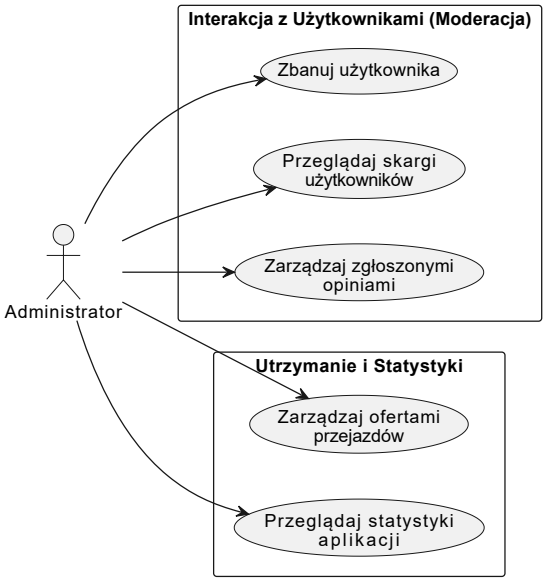
2.1.2 Użytkownik - Pasażer

- Pasażer może przeglądać opublikowane trasy, aby znaleźć swojego kierowce
- Pasażer może wysłać prośbę o dołączenie do przejazdu, aby dołączyć do przejazdu
- Pasażer może wystawić opinię o kierowcy, aby inni pasażerowie wiedzieli więcej o kierowcy
- Pasażer może zapłacić przez aplikację, aby nie musieć rozliczać się gotówką



2.2 Administrator

- Administrator może przeglądać zgłoszenia od użytkowników, aby monitorować stan aplikacji i społeczności
- Administrator może zablokować konto użytkownika, aby zapobiegać nadużyciom w aplikacji
- Administrator może zarządzać opublikowanymi ofertami przejazdów, aby likwidować nadużycia aplikacji
- Administrator może monitorować telemetrie aplikacji, aby przewidywać zapotrzebowanie i wymagania systemu
- Administrator może zarządzać opublikowanymi opiniami, aby zapobiegać nadużyciom w aplikacji

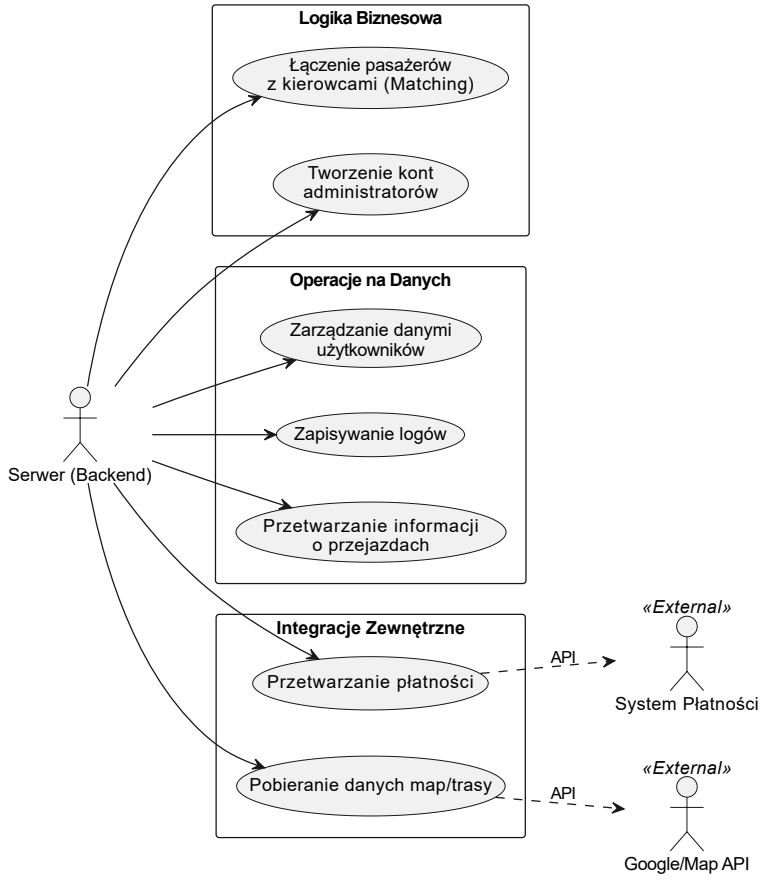


3 Funkcjonalności dostarczane przez aplikację

3.1 Serwer

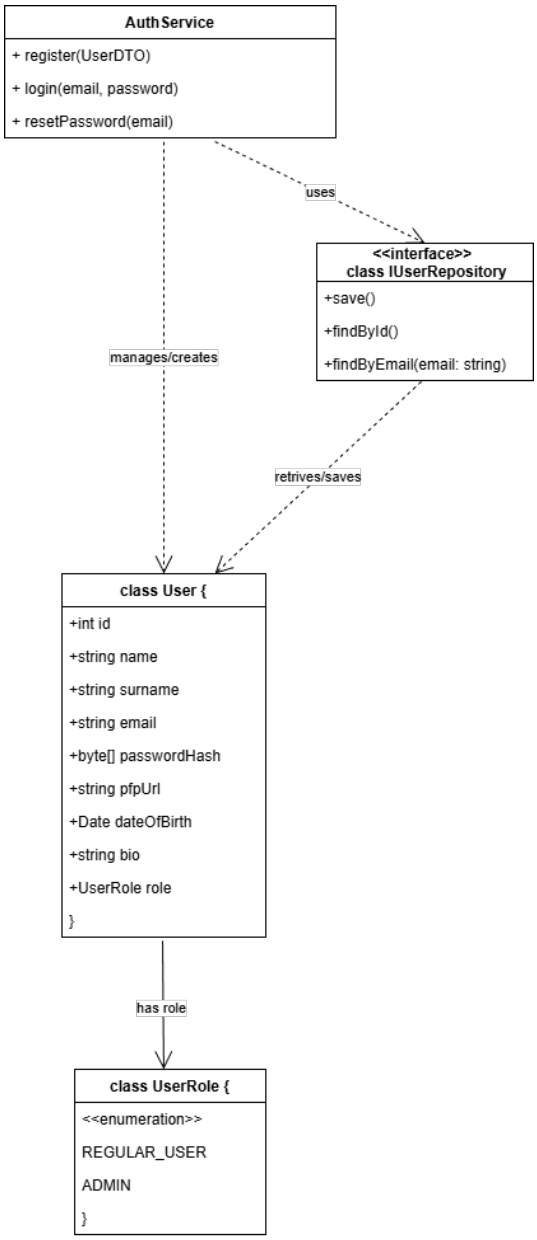
Celem serwera jest zapewnienie powyższych funkcjonalności dla aktorów aplikacji. Serwer będzie umożliwiał:

- Połączenie między użytkownikami
- Przechowanie informacji o przejazdach - trasach, kierowcach oraz pasażerach
- Przechowanie informacji o użytkownikach
- Przetwarzanie płatności przez zewnętrzny system bankowy
- Integracje z Google Maps API w celu dostarczenia dodatkowych danych geo-lokalizacyjnych
- Logowanie błędów oraz aktywności w aplikacji


Logika Biznesowa
Łączenie pasażerów z kierowcami (Matching)
Tworzenie kont administratorów
Operacje na Danych
Zarządzanie danymi użytkowników
Zapisywanie logów
Przetwarzanie informacji o przejazdach
Serwer (Backend)
Integracje Zewnętrzne
Przetwarzanie płatności
Pobieranie danych map/trasy
API
API
«External»
System Płatności
«External»
Google/Map API

4 Klasy

4.1 User

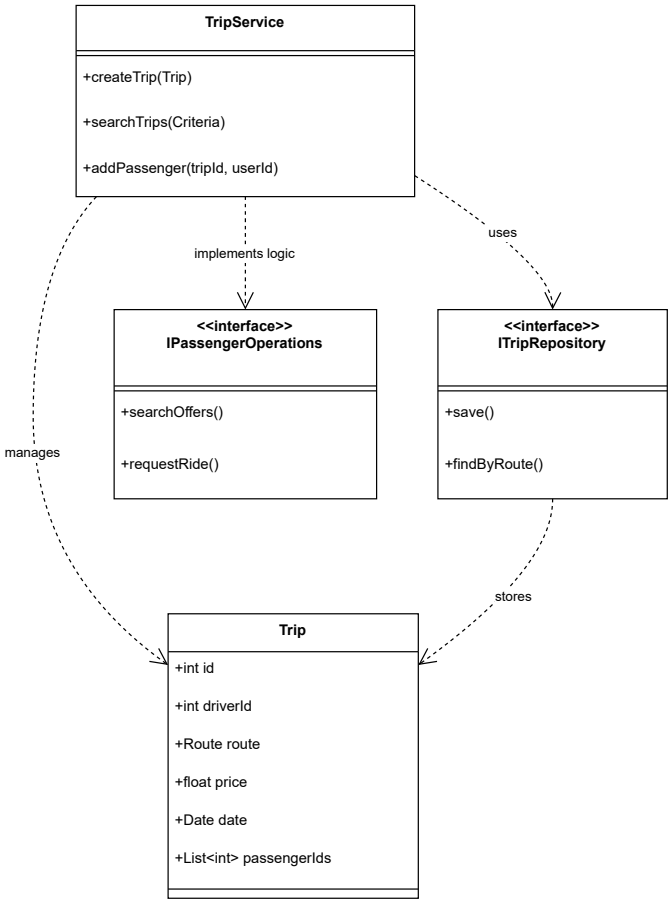


Reprezentuje zarejestrowanego użytkownika systemu. Klasa ta przechowuje kluczowe informacje tożsamościowe, dane uwierzytelniające oraz profilowe. Stanowi punkt wyjścia dla określania ról w systemie (Kierowca, Pasażer, Administrator).

- `id` – Unikalny identyfikator użytkownika w systemie (klucz główny).
- `name` – Imię użytkownika.
- `surname` – Nazwisko użytkownika.
- `email` – Adres email służący do logowania i komunikacji.

- `passwordHash` – Zaszyfrowany skrót hasła.
- `pfp` – Zdjęcie profilowe użytkownika (przechowywane jako plik lub ścieżka do zasobu).
- `dateOfBirth` – Data urodzenia, wymagana do weryfikacji wieku (np. czy użytkownik jest pełnoletni).
- `bio` – Krótki opis tekstowy (`"o mnie"`) widoczny dla innych użytkowników.
- `role` – Rola w systemie określająca uprawnienia (np. `REGULAR_USER`, `ADMIN`).

4.2 Trip

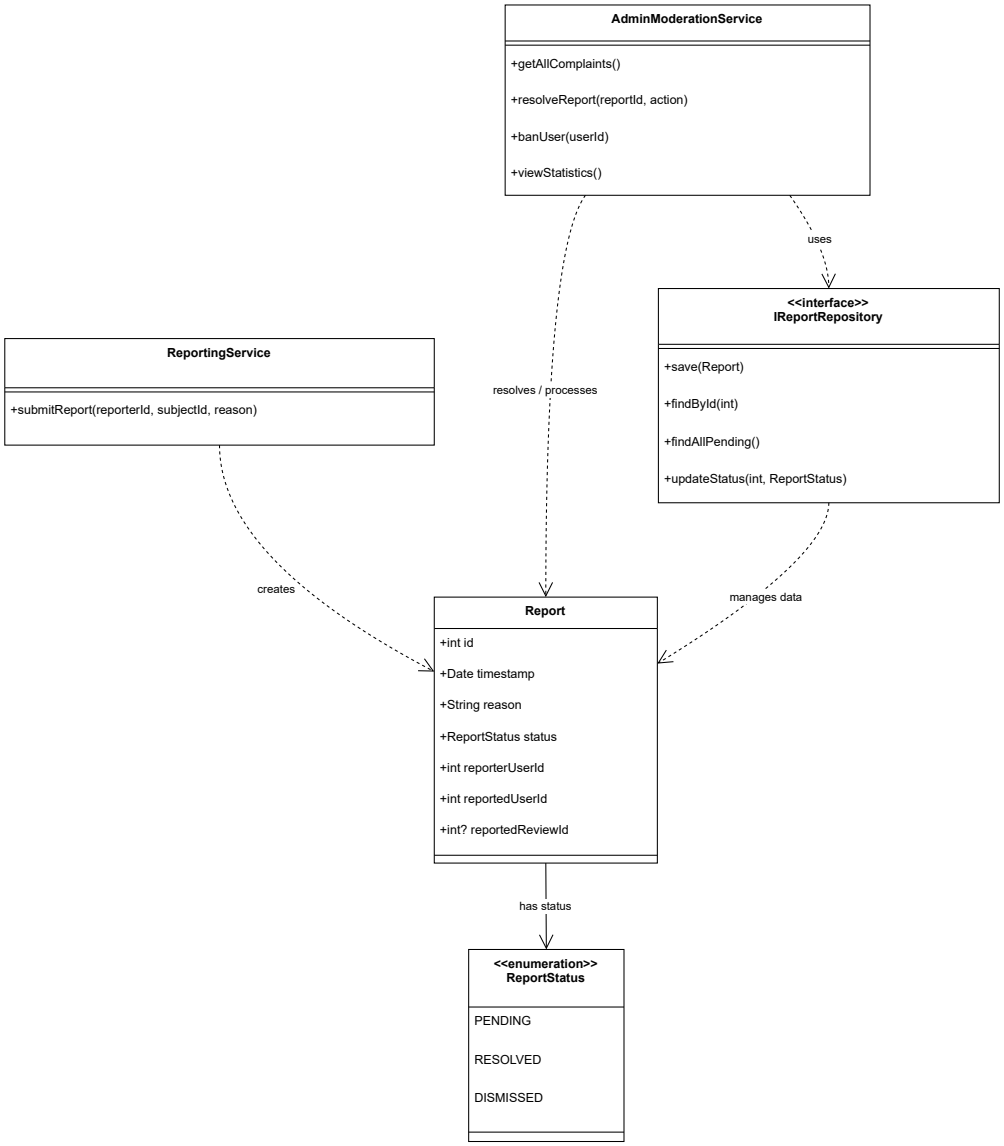


Główna encja biznesowa aplikacji, definiująca ofertę wspólnego przejazdu. Agreguje informacje o trasie, terminie, kosztach oraz uczestnikach podróży (kierowcy i pasażerach).

- `id` – Unikalny identyfikator oferty przejazdu.
- `driverId` – Identyfikator użytkownika (kierowcy), który utworzył ofertę.
- `route` – Obiekt lub struktura definiująca trasę (punkt początkowy, końcowy oraz ewentualne punkty pośrednie).
- `price` – Cena za przejazd dla jednego pasażera.

- `date` – Data i godzina planowanego rozpoczęcia podróży.
- `passengerIds` – Lista identyfikatorów użytkowników, którzy zostali zaakceptowani jako pasażerowie.

4.3 Report

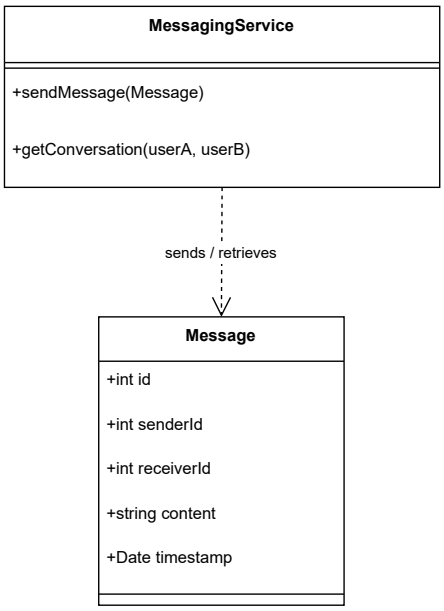


Klasa wspierająca procesy moderacyjne i bezpieczeństwo. Służy do rejestrowania naruszeń regulaminu lub niewłaściwych zachowań, które są następnie przetwarzane przez administratora systemu.

- `id` – Unikalny identyfikator zgłoszenia.
- `timestamp` – Data i czas utworzenia zgłoszenia.
- `reason` – Powód zgłoszenia wybrany z listy lub wpisany przez użytkownika.

- `status` – Aktualny status procedowania zgłoszenia (np. `PENDING`, `RESOLVED`).
- `reporterUserId` – Identyfikator użytkownika dokonującego zgłoszenia.
- `reportedUserId` – Identyfikator użytkownika, który jest zgłaszany.
- `reportedReviewId` – (Opcjonalne) Identyfikator konkretnej opinii, jeśli zgłoszenie dotyczy obraźliwej recenzji.

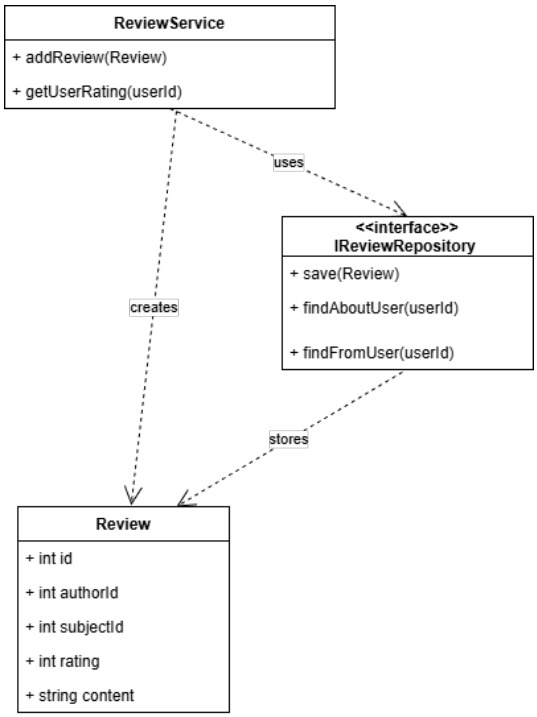
4.4 Message



Klasa odpowiedzialna za system komunikacji wewnętrznej. Umożliwia wymianę informacji tekstowych pomiędzy użytkownikami w celu ustalenia szczegółów przejazdu.

- `id` – Unikalny identyfikator wiadomości.
- `senderId` – Identyfikator użytkownika wysyłającego wiadomość.
- `receiverId` – Identyfikator użytkownika, do którego wiadomość jest adresowana.
- `content` – Treść tekstowa wiadomości.
- `timestamp` – Data i czas wysłania wiadomości.

4.5 Review



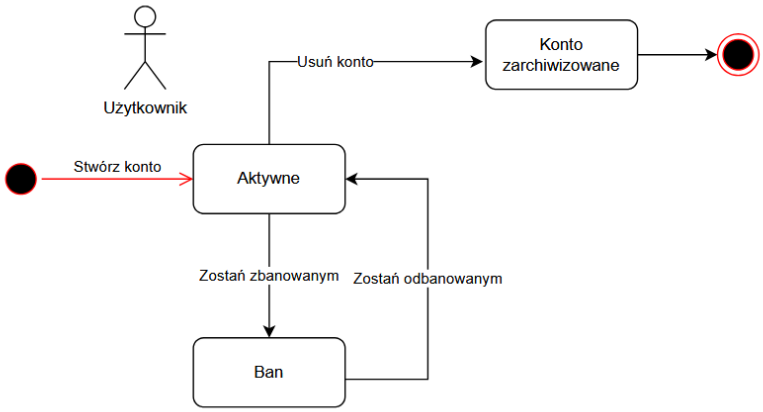
Przechowuje ocenę oraz komentarz wystawiony przez użytkownika po zakończonym przejeździe, co pozwala na weryfikację wiarygodności kierowców i pasażerów.

- `id` – Unikalny identyfikator opinii.
- `authorId` – Identyfikator użytkownika wystawiającego opinię.
- `subjectId` – Identyfikator użytkownika (kierowcy lub pasażera), którego dotyczy opinia.
- `rating` – Ocena liczbowa (np. w skali 1-5).
- `content` – Opcjonalny komentarz tekstowy uzasadniający ocenę.

5 Diagram stanów

5.1 Użytkownik

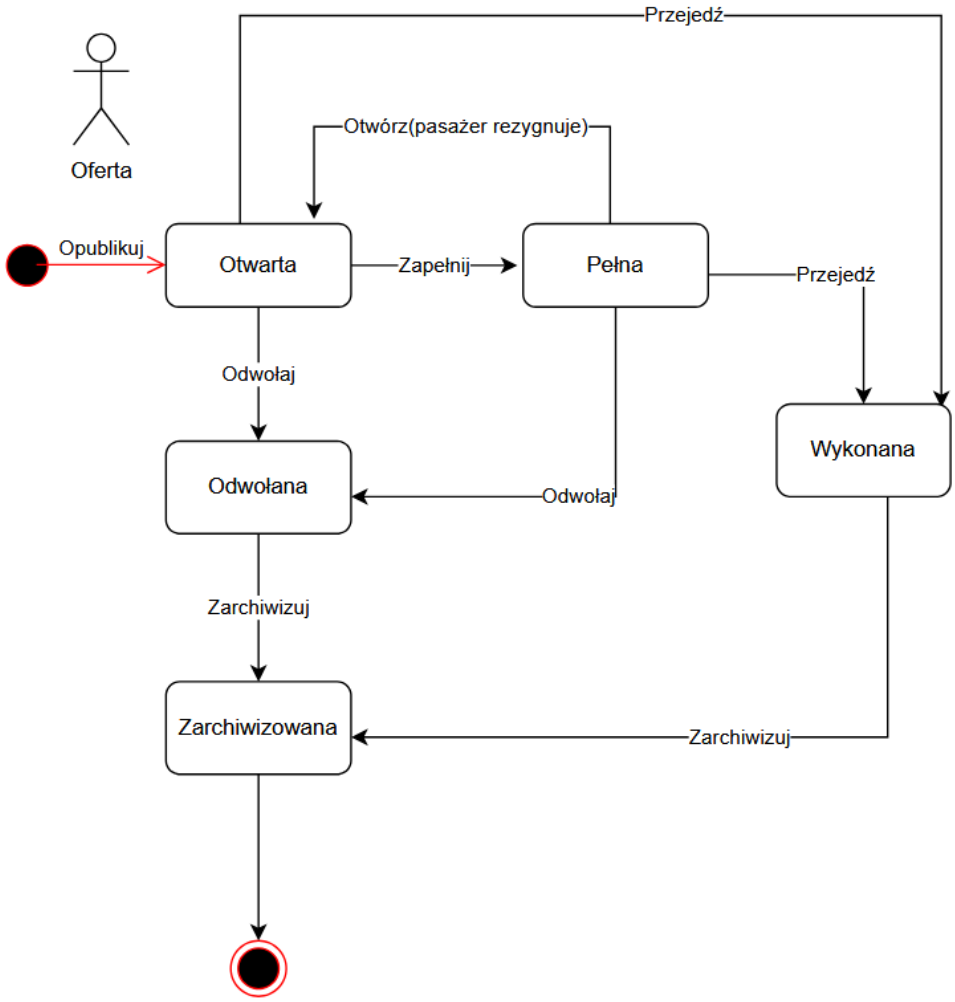
Klasa użytkownika powstaje w wyniku stworzenia konta. Bezpośrednio po stworzeniu konta jest ono aktywne. Następnie może ono zostać zablokowane jeżeli jego aktywność jest podejrzana bądź destrukcyjna dla platformy. Podobnie aktywne konto może zostać zarchiwizowane a dalej usunięte.



5.2 Oferta

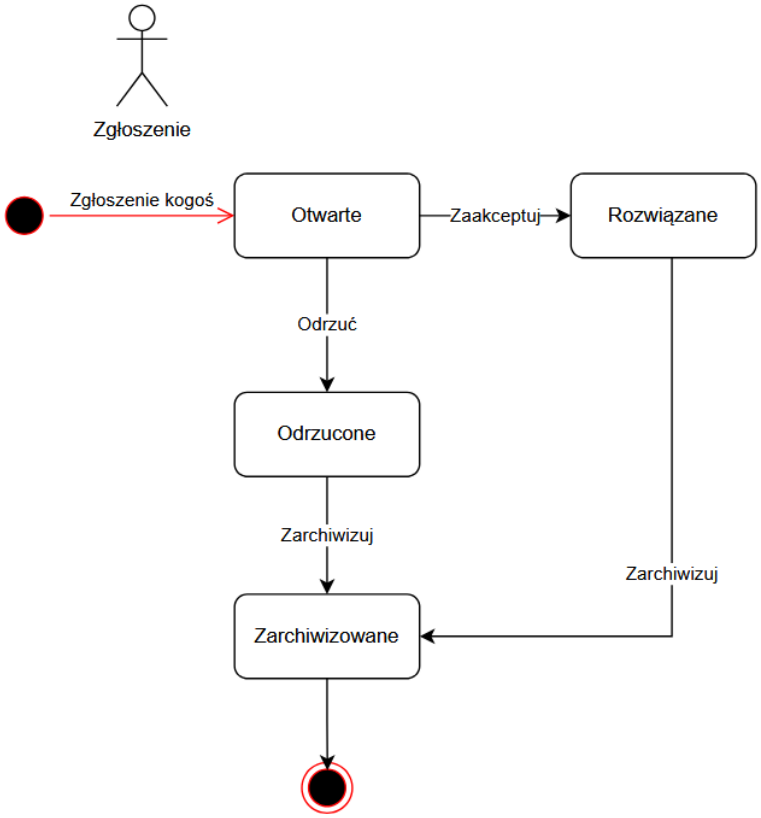
Klasa oferty powstaje w wyniku publikacji trasy przez kierowcę. Następnie oferta może być w kilku stanach:

- Aktywna - bezpośrednio po utworzeniu, pozostaje w tym stanie o ile są jeszcze miejsca w samochodzie kierowcy
- Pełna - gdy liczba pasażerów jest równa liczbie miejsc w samochodzie kierowcy
- Odwołana - gdy kierowca zadecyduje, że nie będzie realizować przejazdu
- Wykonana - gdy przejazd został zrealizowany
- Zarchiwizowana - gdy oferta została odwołana bądź wykonana i pozostaje w systemie w celu zbierania statystyk



5.3 Zgłosznie

Klasa zgłosznia powstaje gdy użytkownik je złoży. Po utworzeniu zgłoszenie jest otwarte i oczekuje na rozpatrzenie. Administrator może albo zaakceptować (tj. rozwiązać problem dotyczący zgłoszenia) bądź odrzucić. Następnie zgłoszenie jest archwizowane w celu zbierania statystyk.



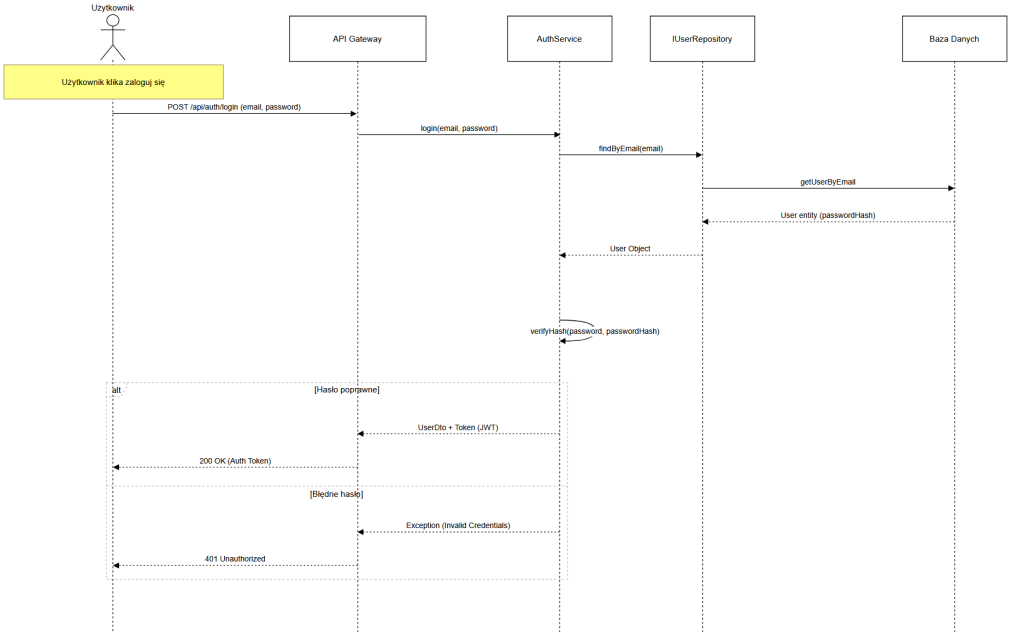
6 Komunikacja w systemie

Wymiana danych w systemie PullApp odbywa się w oparciu o architekturę klient-serwer przy użyciu protokołu HTTP/REST. Komunikacja jest zabezpieczona protokołem HTTPS, a autoryzacja realizowana jest poprzez mechanizm tokenów JWT (JSON Web Token).

6.1 Sekwencje wymiany wiadomości przy zadanym rodzaju działania

6.1.1 Proces logowania użytkownika

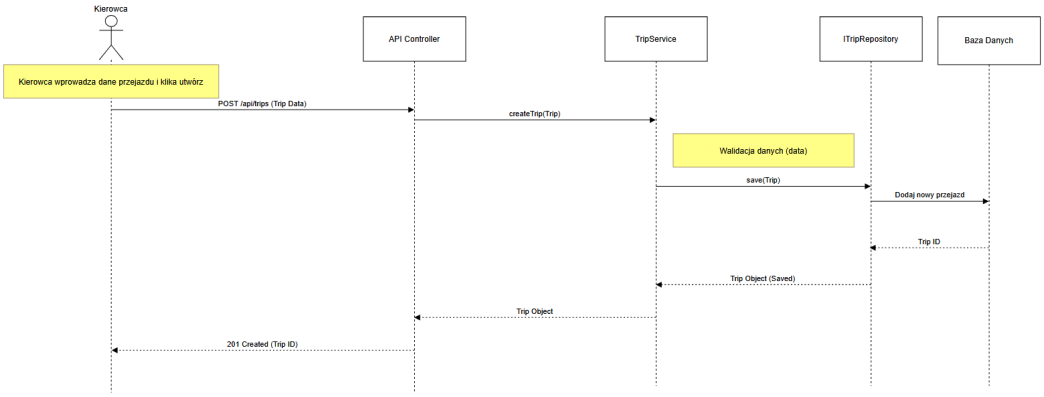
Diagram przedstawia proces uwierzytelniania. Żądanie trafia przez API Gateway do serwisu autoryzacyjnego. System pobiera dane użytkownika (w tym hash hasła) z bazy, weryfikuje poprawność danych i zwraca token JWT (kod 200) lub błąd autoryzacji (kod 401).



Rysunek 1: Diagram sekwencji: Logowanie użytkownika

6.1.2 Publikacja nowego przejazdu

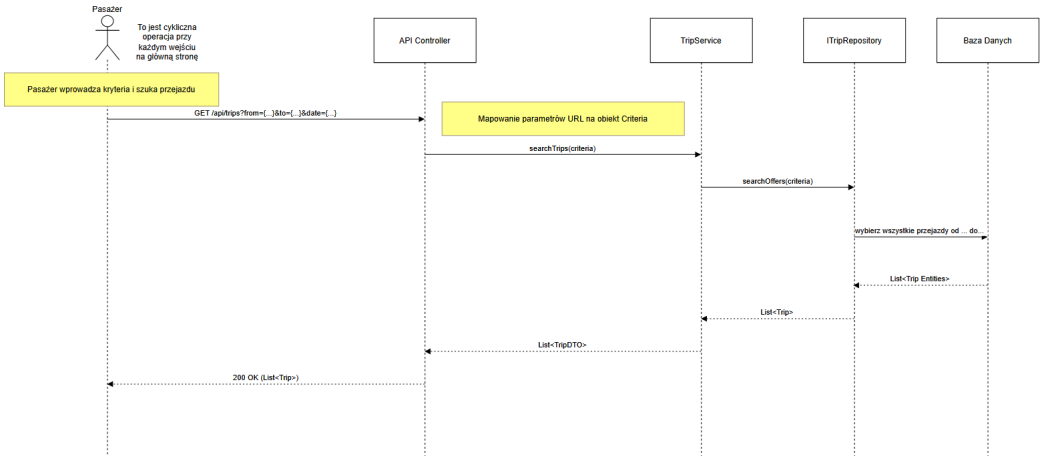
Kierowca przesyła dane nowej oferty. `TripService` waliduje dane, a następnie zapisuje obiekt przejazdu w bazie danych. Operacja kończy się zwróceniem statusu `201 Created` wraz z ID utworzonego zasobu.



Rysunek 2: Diagram sekwencji: Tworzenie przejazdu

6.1.3 Wyszukiwanie ofert

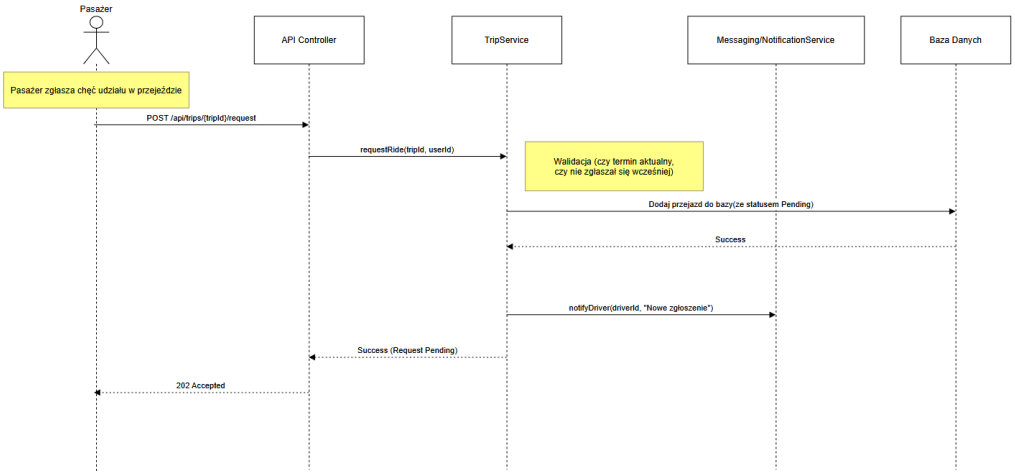
Pasażer definiuje kryteria (trasa, data), które są mapowane na obiekt `Criteria`. Repozytorium filtruje oferty w bazie i zwraca listę pasujących encji, które są konwertowane na obiekty DTO i przesyłane do klienta.



Rysunek 3: Diagram sekwencji: Wyszukiwanie przejazdów

6.1.4 Rezerwacja miejsca (Zgłoszenie)

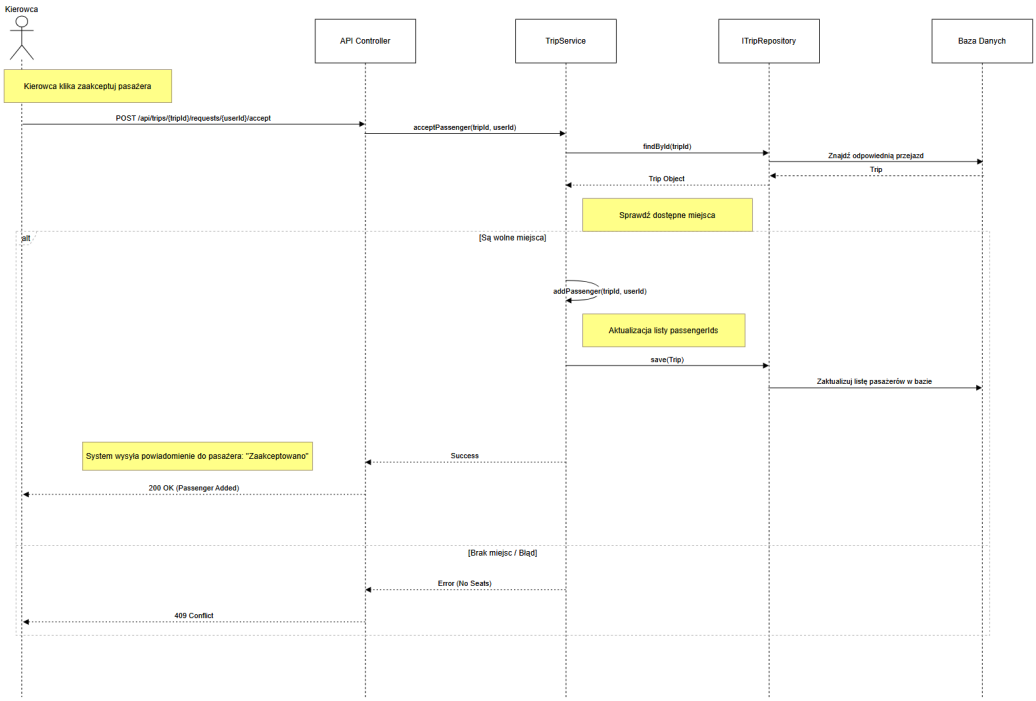
Pasażer zgłasza chęć udziału w przejeździe. System weryfikuje termin i dostępność. Po walidacji tworzony jest wpis w bazie ze statusem `Pending`, a kierowca otrzymuje powiadomienie.



Rysunek 4: Diagram sekwencji: Zgłoszenie pasażera

6.1.5 Akceptacja pasażera

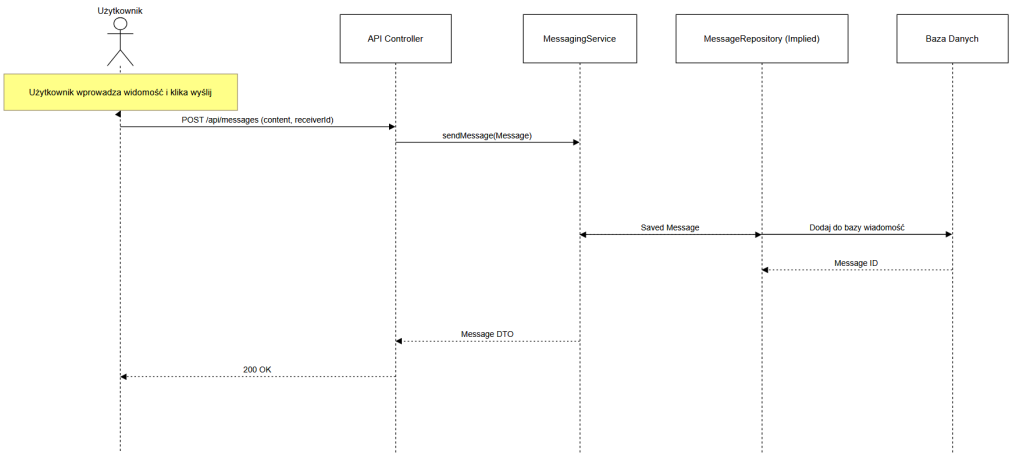
Kierowca zatwierdza kandydata. Serwis sprawdza dostępność miejsc (sekcja krytyczna). Jeśli miejsce jest dostępne, pasażer jest dodawany do listy, a stan przejazdu aktualizowany. W przeciwnym razie zwracany jest błąd `409 Conflict`.



Rysunek 5: Diagram sekwencji: Akceptacja pasażera

6.1.6 Wymiana wiadomości

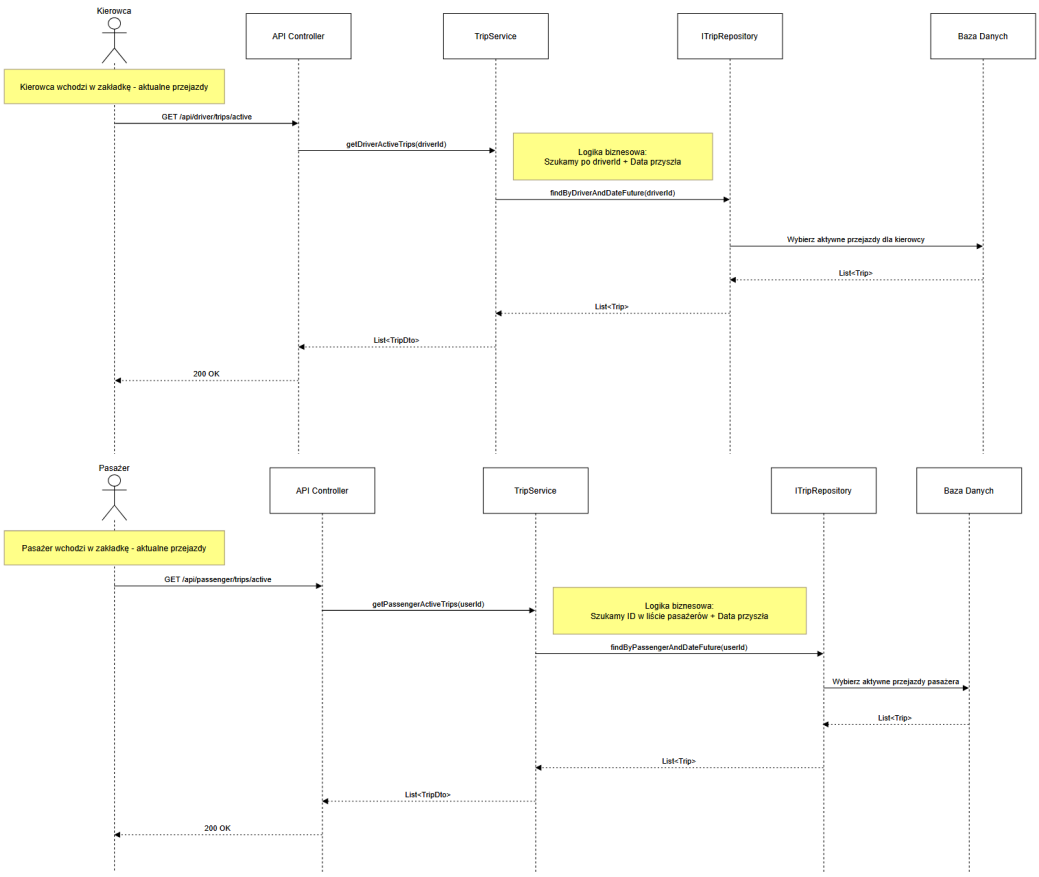
Użytkownik wysyła wiadomość do innego użytkownika. `MessagingService` przetwarza treść i zapisuje ją w bazie danych, zwracając potwierdzenie nadawcy.



Rysunek 6: Diagram sekwencji: Wysyłanie wiadomości

6.1.7 Pobieranie aktualnych przejazdów

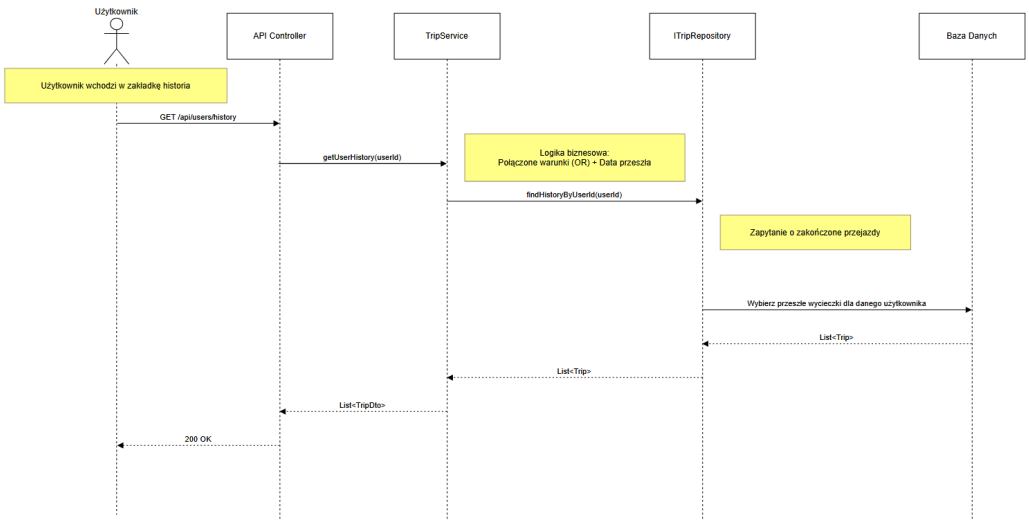
Diagram obrazuje pobieranie aktywnych ofert dla kierowcy i pasażera. System filtruje przejazdy powiązane z użytkownikiem, biorąc pod uwagę tylko te z datą przyszłą.



Rysunek 7: Diagram sekwencji: Aktualne przejazdy

6.1.8 Historia przejazdów

Użytkownik pobiera archiwum podróży. Logika biznesowa łączy warunki uczestnictwa (kierowca lub pasażer) oraz filtruje wyniki pod kątem zakończonych przejazdów (data przeszła).



Rysunek 8: Diagram sekwencji: Historia przejazdów

6.2 Endpointy API serwera

Interfejs programistyczny aplikacji został zaprojektowany zgodnie z zasadami REST. Poniższa tabela prezentuje zestawienie głównych punktów końcowych (endpoints) dostępnych dla aplikacji klienckich.

Tabela 1: Specyfikacja wybranych endpointów API

Metoda	Endpoint	Opis funkcjonalności	Parametry
Moduł Uwierzytelniania (Auth)			
POST	/api/auth/login	Logowanie standardowe	email, password
POST	/api/auth/google	Logowanie przez Google	id_token
Moduł Przejazdów (Trips)			
POST	/api/trips	Publikacja nowego przejazdu	route, date, price
GET	/api/trips/{id}	Pobranie szczegółów przejazdu	id (path)
GET	/api/trips	Wyszukiwanie (z filtrami)	from, to, date
Moduł Rezerwacji (Booking)			
POST	/api/trips/{id}/request	Rezerwacja miejsca	userId, seats
POST	.../requests/{id}/accept	Akceptacja pasażera	userId
Moduł Wiadomości (Messaging)			
POST	/api/messages	Wysłanie wiadomości	content, receiverId
GET	/api/messages/{userId}	Pobranie historii rozmowy	userId (path)
Moduł Raportowania (Reports)			
POST	/api/reports	Zgłoszenie naruszenia	subjectId, reason

6.3 Struktura komunikatów (DTO)

Poniżej przedstawiono strukturę kluczowych obiektów przesyłanych w ciele żądań i odpowiedzi. Obiekty te (Data Transfer Objects) definiują kontrakt danych między frontendem a backendem.

1. Obiekt Autoryzacji (Auth Response) Zwracany po poprawnym zalogowaniu.

Listing 1: Struktura odpowiedzi logowania

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI...",
  "expiresIn": 3600,
  "user": {
    "id": 105,
    "firstName": "Jan",
    "role": "USER"
  }
}
```

2. Obiekt Przejazdu (Trip Details) Wykorzystywany przy wyświetlaniu szczegółów oferty.

Listing 2: Struktura szczegółów przejazdu

```
{
  "id": 5501,
  "routeTitle": "Warszawa - Krakow",
  "GPSstart": "41.40338N,-2.17403E",
  "GPSend": "48.8548N,-2.17403E",
  "date": "2023-12-24T08:00:00Z",
  "price": 45.50,
  "driver": {
    "id": 12,
    "firstName": "Marek",
    "averageRating": 4.8
  }
}
```

3. Obiekt Wiadomości (Message) Struktura pojedynczej wiadomości na czacie.

Listing 3: Struktura wiadomości

```
{
  "id": 991,
  "senderId": 105,
  "receiverId": 202,
  "content": "Czesc, czy miejsce nadal aktualne?",
  "timestamp": "2023-12-23T18:30:00Z"
}
```